Lab 1：Data Lab 实验报告

姓名：苏丹

学号：24300240013

实验环境：Ubuntu 虚拟机、GCC、Python 3、Git

个人仓库：https://github.com/24300240013-fudan-edu-cn/DataLab

一、实验目的

通过完成受限的整数运算与浮点数位级操作，理解 32 位二进制补码、掩码、移位、溢出判断和 IEEE 754 单精度浮点数的表示及舍入规则。

二、实验环境与完成过程

1. 获取实验仓库

使用课程 DataLab 模板创建个人仓库，通过 SSH 克隆到 Ubuntu 虚拟机。

git clone git@github.com:24300240013-fudan-edu-cn/DataLab.git cd ~/DataLab

2. 实现函数

按照 bits.c 中的要求完成 P1—P19，仅修改对应函数体，没有修改函数名、参数、返回类型、测试程序或评分配置。

整数题遵循 32 位补码与位向量模型，仅使用允许的常数、int 类型变量和运算符。浮点题通过整数运算处理浮点数编码，不直接使用浮点类型或强制类型转换。

3. 编译与测试

修改代码后重新构建：

make clean && make all

运行规则检查：

./check_ops.py bits.c

运行完整正确性测试：

./btest

4. 提交代码

仅将 bits.c 加入暂存区，提交并推送到 main 分支，避免提交编译产物。

git add bits.c git commit -m "feat: complete DataLab P1-P19 solutions"

git push origin main

三、测试结果

1. 运算符与操作数检查

19 个函数均通过规则检查，输出结尾为：

All 19 functions passed operator checks.

各函数操作数如下，斜杠前为实际操作数，后为允许的最大操作数：

P1 signMask：1/2

P2 bitXor：7/8

P3 negativePart：4/6

P4 copyByteWithin：9/12

P5 logicalShift：6/20

P6 swapNibblePairs：9/18

P7 secondLowestZeroBit：5/8

P8 oddParity：12/56

P9 rotateRightBits：12/16

P10 roundEvenPow2：12/24

P11 midpointTowardFirst：19/32

P12 isBetweenEitherOrder：30/48

P13 mul5Sat：17/30

P14 classifyAdd3：23/52

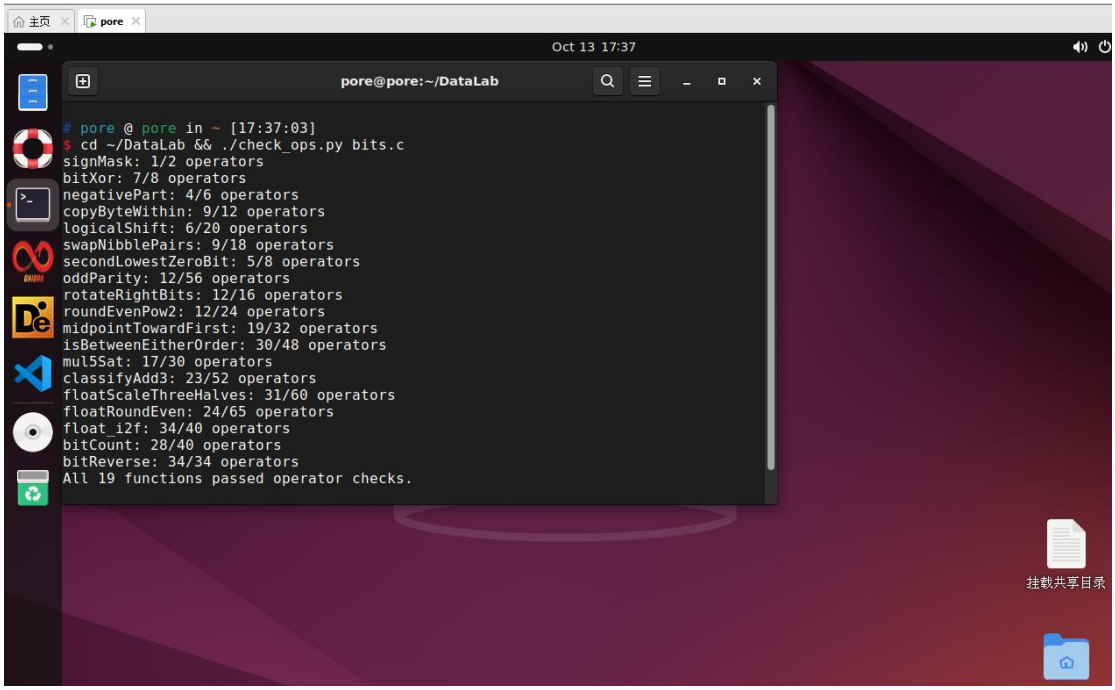
P15 floatScaleThreeHalves：31/60

P16 floatRoundEven：24/65

P17 float_i2f：34/40

P18 bitCount：28/40

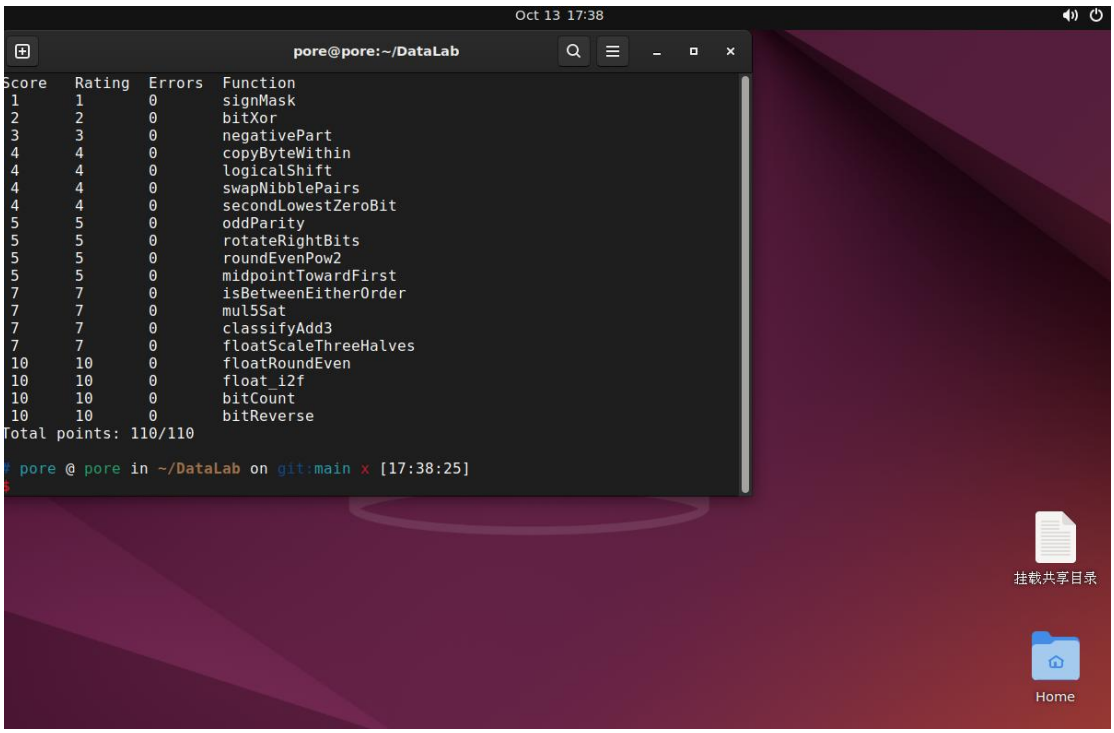
P19 bitReverse：34/34



2. 本地正确性测试

完整测试中，各函数的 Errors 均为 0，最终结果为：

Total points: 110/110

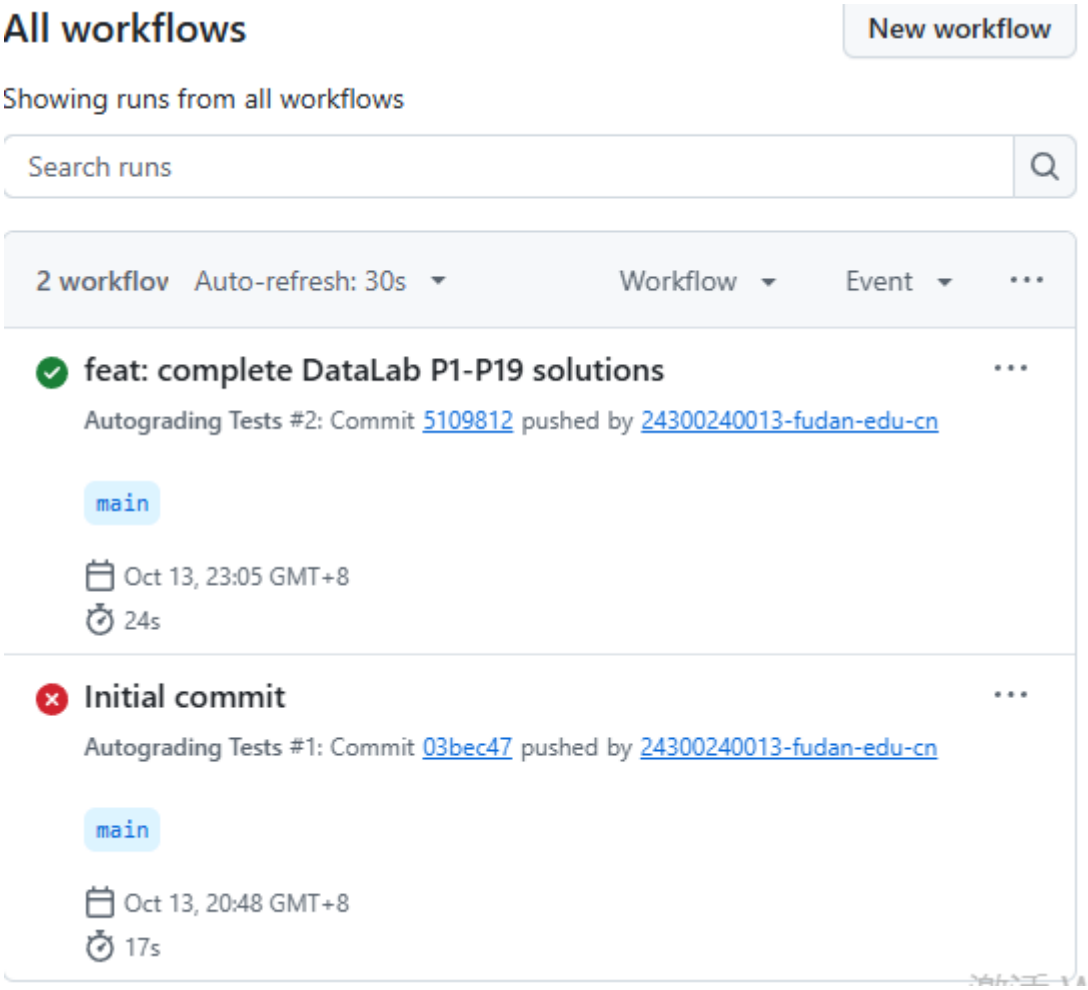


3. GitHub Actions 自动评测

提交编号：5109812 提交说明：feat: complete DataLab P1-P19 solutions

最新提交的自动评测工作流运行成功，评分结果为：

Points 110/110



四、各函数实现思路

1. signMask

将整数 1 左移 31 位，使最高位为 1，其余位为 0，得到 0x80000000。该操作遵循实验规定的 32 位位向量模型。

核心表达式：

1 << 31

2. bitXor

异或要求对应位中恰好有一个 1。根据德摩根律，“至少有一个 1”可以写成 ~(~x & ~y)，“不能同时为 1”可以写成 ~(x & y)。将两者相与，即可仅使用 ~ 和 & 实现异或。

核心表达式：

~(~x & ~y) & ~(x & y)

3. negativePart

通过 x >> 31 获取符号掩码：负数得到全 1，非负数得到全 0。使用 ~x + 1 构造 x 的补码相反数，再与符号掩码相与。

因此，负数返回其相反数，非负数返回 0。对于 INT_MIN，按照实验规定保留低 32 位，结果仍为 INT_MIN。

核心表达式

(x >> 31) & (~x + 1)

4. copyByteWithin

将字节编号左移 3 位，转换为位偏移。将 x 右移源偏移，再与 255 相与，提取源字节。

随后构造目标位置的字节掩码，清空原目标字节，再将源字节移入该位置。其他字节不变，源位置与目标位置相同时也能正确处理。

5. logicalShift

有符号数的算术右移会补符号位，而逻辑右移需要补 0。因此先执行算术右移，再使用低 32-n 位为 1 的掩码清除高位。

掩码表达式为：

~(((1 << 31) >> n) << 1)

该构造在 n=0 时得到全 1，在 n=31 时只保留最低位，且不会出现移位 32 位。

6. swapNibblePairs

构造 0x0F0F0F0F 掩码，提取每个字节的低 4 位，并左移 4 位。将原数右移 4 位后再与该掩码相与，提取原来的高 4 位。

将两部分按位或，即可交换每个字节内部的高低 4 位。较大的掩码由允许的小常数通过移位和按位或构造。

7. secondLowestZeroBit

利用 x | (x + 1) 将最低的一个 0 置为 1，得到 y，其余位保持不变。

再通过 ~y & (y + 1) 提取 y 中最低的 0。该位置就是原数中第二低的 0。

当原数不足两个 0 时，y 为全 1，~y 为 0，因此返回 0。

8. oddParity

利用异或汇总各位的奇偶信息。依次将 x 与其右移 16、8、4、2、1 位的结果异或，最终最低位表示原数中 1 的数量是否为奇数。

题目要求偶数个 1 返回 1，奇数个 1 返回 0，因此取最低位后再进行逻辑非。

返回表达式：

!(x & 1)

9. rotateRightBits

首先通过 n & 31 将循环位数归一化，因为循环 32 位等价于不移动。

使用掩码实现逻辑右移，再将原数左移对应位数，将低位移到高位，最后按位或。

左移量使用 (~k + 1) & 31 构造：k>0 时为 32-k，k=0 时为 0，避免出现移位 32 位。

10. roundEvenPow2

令 unit=2^n。先构造偏置 unit/2-1，再加上当前商的最低位，最后清除低 n 位。

当输入恰好位于两个倍数中间时： 当前商为偶数，不进位； 当前商为奇数，进位到偶数商。

不在中点时，该偏置也能使结果舍入到最近的倍数。代码中用 ~0 表示-1，避免使用整数题禁止的减号。

11. midpointTowardFirst

使用下面的公式计算向下取整的中点，避免直接计算 x+y 引起溢出：

(x & y) + ((x ^ y) >> 1)

当 x、y 最低位不同时，它们的和为奇数，中点带有 0.5。若此时 x>y，则将向

下取整的结果加 1，使结果向第一个参数 x 靠近。

判断 x>y 时区分同号与异号：异号时根据符号判断，同号时根据 y-x 的符号判断。

12. isBetweenEitherOrder

分别判断 x<a 和 x<b。若两者结果不同，说明 x 位于两个端点之间。再补充 x=a 或 x=b 的判断，实现包含端点的闭区间判断。

为避免减法溢出，比较时先检查符号：异号时根据 x 的符号判断，同号时根据差值的符号判断。因此也能处理 INT_MIN、INT_MAX 等边界值。

13. mul5Sat

依次计算 2x、4x 和 5x，并检查这些结果是否与 x 的符号不同。只要某个阶段发生符号变化，就说明数学上的 5x 已经超出允许范围。

不能只检查最终结果，因为多次溢出可能使最终符号重新与输入一致。

溢出时根据 x 的符号选择 INT_MAX 或 INT_MIN，未溢出时返回 5x。最终通过掩码选择结果，不使用条件语句。

14. classifyAdd3

将每个输入分解为带符号的高位部分和低 30 位部分：

x = (x >> 30) * 2^30 + (x & 0x3FFFFFFF)

上式用于说明数学分解，实际代码不使用乘法。

先对低 30 位求和，提取向高位产生的进位，再与三个高位部分相加。

若合并后的高位部分在[-2,1]内，则总和能够由 32 位有符号整数表示；若大于等于 2，则正溢出；若小于-2，则负溢出。分别返回 0、1 或-1。

15. floatScaleThreeHalves

拆解符号位、指数域和小数域，为规格化数补上隐藏的最高位 1。将有效数作为整数乘 3，再通过右移实现除 2 及规格化。

根据被舍弃部分与半程值的比较，执行最接近偶数舍入。若舍入使有效数产生进

位，则继续调整指数。

对非规格化数、正负零、指数溢出分别处理；NaN 和无穷大原样返回。使用整数有效数完成计算后统一舍入，避免分步浮点舍入产生误差。

16. floatRoundEven

按指数域分类处理：

绝对值小于 0.5 时，返回带输入符号的零。

绝对值在[0.5,1)内时，恰好 0.5 舍入到零，其余值舍入到带相同符号的 1。

指数足够大的有限数已经没有小数部分，直接返回；NaN 和无穷大也原样返回。

对于其余数值，清除表示小数部分的位，根据被清除部分是否超过半程，以及整数部分的奇偶性决定是否进位。返回值仍然是单精度浮点数编码。

17. float_i2f

提取输入符号，并用 unsigned 保存绝对值，避免对 INT_MIN 直接进行有符号求负。

通过右移寻找最高位 1 的位置，从而确定指数。若有效位不超过 24 位，则左移对齐，可精确表示；否则保留最高 24 位，并根据被舍弃位执行最接近偶数舍入。舍入可能导致有效数进位，需要同步调整指数。最后拼接符号、指数及小数域。

18. bitCount

采用分组累加法，先统计每 2 位中 1 的数量，再合并为每 4 位、每 8 位的计数，最后汇总整个 32 位。

使用 0x55555555、0x33333333、0x0F0F0F0F 等掩码隔离各组。这些掩码由合法的小常数构造。

最终计数范围为 0～32，保留结果的低 6 位即可。

19. bitReverse

依次交换相邻的 1 位、2 位组和 4 位组，使每个字节内部的位序反转。之后再反转四个字节的排列顺序，完成整个 32 位的反转。

为控制操作数，利用移位和异或从一个掩码生成其他掩码。最终通过规则检查，操作数为 34，恰好达到题目上限。

五、实验总结

本实验的关键是区分二进制位模式和对应的数值含义。同一位模式在有符号整数、无符号整数和浮点数解释下具有不同意义。

整数题需要重点考虑符号扩展、溢出、INT_MIN 和边界移位。浮点题需要区分规格化数、非规格化数、正负零、无穷大与 NaN，并正确实现最接近偶数舍入。

规则检查与正确性测试不能相互替代。规则检查通过只说明代码形式符合要求，正确性测试通过才说明测试范围内的输出符合预期。本次 19 题均通过规则检查，本地及线上自动测试均得到 110/110。

六、参考资料与辅助使用说明

课程《Lab 1：Data Lab 实验指南》。

课程 DataLab 模板仓库及 bits.c 中的编码规则、函数注释和输入输出约定。

https://github.com/ICS-26Fall-FDU/DataLab

OpenAI ChatGPT